

Ключник

Курс по Telegram-ботам: от пустой папки до бота на сервере

Закрытый курс сообщества. Файл не пересылаем за пределы чата.

Содержание

- Урок 0. Правила игры
- Урок 1. Скелет: первый бот за 15 минут
- Урок 2. Кнопки: двери Ключника
- Урок 3. Диалоги: страж у двери
- Урок 4. Память: SQLite
- Урок 5. Middleware и доступ хозяина
- Урок 6. Бот 24/7: сервер и systemd
- Урок 7. Финал: пропуск у Привратника

Урок 0. Правила игры

За 7 уроков ты соберёшь своего Telegram-бота — маленький квест с дверями, стражем и памятью. В конце бот сам заработает тебе пропуск в чат выпускников.

Как это работает

- В каждом уроке с 1 по 6 спрятан ключ. Он появляется, только когда ты делаешь практику: бот что-то ответит, код что-то напечатает, сервер выдаст ошибку.
- Записывай ключи в файл `keys.txt` по порядку и точно как их выдали: с тем же регистром, без пробелов и кавычек.
- В финале твой бот посчитает пропуск из шести ключей, твоего Telegram ID и слова от Привратника — бота нашего сообщества. Ссылка на Привратника — в закреплённом посте.
- Пропуск личный. Чужой тебе не подойдёт, твой не подойдёт другим. Один бот проводит через дверь одного человека.
- Ключи в чате не выкладываем. Помогать друг другу с кодом — можно и нужно.

Что нужно

- Python 3.10 или новее и любой редактор (подойдёт VS Code).
- Базовый Python: переменные, функции, `if`. Классы и `async` объясним по ходу.
- К уроку 6 — сервер на Linux. Хватит самого дешёвого VPS.

Стек курса: `aiogram 3`, `SQLite`, `systemd`. Вопросы — в комментариях.

Урок 1. Скелет: первый бот за 15 минут

Сегодня заводим бота, ставим aiogram и учим его отвечать. В конце — первый ключ.

Шаг 1. Бот у BotFather

1. Открой @BotFather и отправь /newbot.
2. Придумай имя (любое) и username. Username должен заканчиваться на bot, например kluchnik_ivan_bot.
3. BotFather пришлёт токен вида 7395021846:ААН... . Это пароль от бота.

Две вещи про токен:

- Число до двоеточия — ID твоего бота. Запомни это, в финале пригодится.
- Токен никому не показывай и не выкладывай в GitHub. Утёк — отправь BotFather /revoke и получи новый.

Шаг 2. Проект

```
mkdir kluchnik
cd kluchnik
python -m venv venv
```

Активируй окружение. Windows: `venv\Scripts\activate`. macOS и Linux: `source venv/bin/activate`. Затем:

```
pip install aiogram python-dotenv
```

Создай рядом с кодом файл `.env`:

```
BOT_TOKEN=сюда_твой_токен
```

Токен живёт в `.env`, а не в коде. Так он не уедет на GitHub вместе с кодом.

Шаг 3. Код

Файл `bot.py`:

```
import asyncio
import logging
import os

from aiogram import Bot, Dispatcher, F, Router
from aiogram.filters import CommandObject, CommandStart
from aiogram.types import Message
from dotenv import load_dotenv

load_dotenv()
router = Router()
```

```

@router.message(CommandStart(deep_link=True))
async def start_with_word(message: Message, command: CommandObject):
    await message.answer(command.args[::-1])

@router.message(CommandStart())
async def start(message: Message):
    await message.answer("Ключник на посту. Двери пока закрыты.")

@router.message(F.text)
async def echo(message: Message):
    await message.answer(f"Ты сказал: {message.text}")

async def main():
    logging.basicConfig(level=logging.INFO)
    bot = Bot(os.environ["BOT_TOKEN"])
    dp = Dispatcher()
    dp.include_router(router)
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```

Запусти: `python bot.py`. В консоли появится `Start polling`. Напиши боту в Telegram — он ответит. Остановить бота: `Ctrl+C`.

Как это работает

- Бот сам раз за разом спрашивает Telegram «есть новое?». Это называется `polling`. Каждое событие — сообщение, нажатие кнопки — приходит как `update`.
- `Dispatcher` принимает `update` и передаёт его роутерам. `Router` — коробка с хендлерами.
- Хендлер — функция с декоратором. Декоратор говорит, на что она реагирует: `CommandStart()` — на `/start`, `F.text` — на любой текст.
- `async def` и `await` нужны, чтобы бот не замирал, пока ждёт ответа от Telegram. Перед каждым обращением к Telegram пишем `await`.
- Хендлеры проверяются сверху вниз, сообщение забирает первый подходящий. Поэтому эхо, которое ловит любой текст, всегда стоит последним.

Почему `/start` с параметром стоит выше обычного `/start`: `CommandStart()` ловит и «`/start`», и «`/start` что-угодно». Поставь его первым — и до `deep_link=True` очередь не дойдёт.

Ссылка вида `t.me/имя_бота?start=слово` открывает бота и отправляет ему «`/start слово`». Это `deep link`: так делают реферальные ссылки и рекламные метки. Параметр — до 64 символов: латиница, цифры, `_` и `-`. Запись `command.args[::-1]` переворачивает строку задом наперёд.

Ключ 1. Открой ссылку https://t.me/ИМЯ_ТВОЕГО_БОТА?start=7_atorov (username бота без @) и нажми «Запустить». Ответ бота — ключ 1. Запиши его в keys.txt.

Домашка по желанию: пусть эхо отвечает ещё и длиной сообщения.

Урок 2. Кнопки: двери Ключника

Сегодня команда /doors покажет коридор из трёх дверей. В каждую можно постучать или попробовать её открыть.

Два вида кнопок

- Reply-клавиатура заменяет обычную клавиатуру. Нажатие просто отправляет текст.
- Inline-клавиатура висит под сообщением. Нажатие отправляет боту невидимый callback со строкой callback_data. Её мы и используем.

У callback_data есть предел — 64 байта. Кириллица занимает 2 байта на букву, поэтому туда пишем коротко и латиницей.

CallbackData: строки без ручного разбора

Можно склеивать строки руками («door_3_open»), а потом резать их split. aiogram делает это за нас: описываем класс, а он сам превращает его в строку и обратно.

Добавь в импорты:

```
from aiogram.filters import Command, CommandObject, CommandStart
from aiogram.filters.callback_data import CallbackData
from aiogram.types import CallbackQuery, Message
from aiogram.utils.keyboard import InlineKeyboardBuilder
```

Под строкой `router = Router()`:

```
class Door(CallbackData, prefix="kd"):
    room: int
    act: str

def corridor():
    kb = InlineKeyboardBuilder()
    for room in (1, 2, 3):
        kb.button(text=f"Дверь {room}: постучать", callback_data=Door(room=room, act="knock"))
        kb.button(text="открыть", callback_data=Door(room=room, act="open"))
    kb.adjust(2)
    return kb.as_markup()
```

Хендлеры — выше эха:

```
@router.message(Command("doors"))
async def doors(message: Message):
    await message.answer("Коридор. Три двери.", reply_markup=corridor())

@router.callback_query(Door.filter(F.act == "knock"))
async def knock(callback: CallbackQuery, callback_data: Door):
    await callback.answer(f"За дверью {callback_data.room} тихо.")
```

```
@router.callback_query(Door.filter(F.act == "open"))
async def open_door(callback: CallbackQuery, callback_data: Door):
    await callback.answer("Заперто.", show_alert=True)
```

Как это работает

- `kb.adjust(2)` раскладывает кнопки по две в ряд.
- `Door.filter(F.act == "knock")` пропускает к хендлеру только кнопки «постучать». В `callback_data` хендлер получает уже собранный объект, и `room` в нём — число, а не строка.
- `callback.answer()` обязателен. Без него на кнопке будут крутиться часики. С текстом он показывает всплывающую подсказку, с `show_alert=True` — окно с кнопкой ОК.
- Хендлер `/doors` обязан стоять выше эха. Иначе эхо ответит «Ты сказал: `/doors`». Правило курса: эхо — всегда последний хендлер в файле.

Ключ 2. Открой второй терминал в папке проекта, активируй `venv` и выполни: `python -c "from bot import Door; print(Door(room=12, act='knock').pack())"`. Напечатанная строка — ключ 2.

Бот при этом не запустится. Функция `main()` вызывается, только когда файл запускают напрямую, — для этого и нужна строчка `if __name__ == "__main__":`.

Домашка по желанию: посчитай, сколько байт занимает самая длинная `callback_data` в твоём боте. Подсказка: `len(строка.encode())`.

Урок 3. Диалоги: страж у двери

Сегодня за дверью 3 встанет страж. Он спросит, кто ты, а потом потребует пароль. Три ошибки — и он прогонит тебя.

Зачем состояния

Бот получает сообщения по одному и сам не помнит, о чём шла речь. Пришло «Мирон» — это ответ стражу или просто текст? Состояние (FSM, машина состояний) — пометка «этот человек сейчас отвечает на такой-то вопрос».

Добавь в импорты:

```
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
```

Рядом с классом Door:

```
class Guard(StatesGroup):
    waiting_name = State()
    waiting_word = State()
```

Что умеет `state` в хендлере:

- `await state.set_state(...)` — поставить пометку;
- `await state.update_data(...)` и `await state.get_data()` — хранить ответы по ходу диалога;
- `await state.clear()` — снять пометку и стереть данные.

Хендлер с фильтром `Guard.waiting_name` сработает только у того, кто стоит в этом состоянии.

Код

Замени хендлер `open_door`:

```
@router.callback_query(Door.filter(F.act == "open"))
async def open_door(callback: CallbackQuery, callback_data: Door, state: FSMContext):
    if callback_data.room != 3:
        await callback.answer("Занято.", show_alert=True)
        return
    await state.set_state(Guard.waiting_name)
    await callback.message.edit_text("У двери 3 стоит страж.\n— Кто идёт?")
    await callback.answer()
```

Новые хендлеры. `/cancel` ставь выше хендлеров стража, а всё вместе — выше эха:

```
@router.message(Command("cancel"))
async def cancel(message: Message, state: FSMContext):
    await state.clear()
    await message.answer("Ты отошёл от двери.")
```

```

@router.message(Guard.waiting_name, F.text)
async def guard_name(message: Message, state: FSMContext):
    await state.update_data(name=message.text, tries=0)
    await state.set_state(Guard.waiting_word)
    print(await state.get_state())
    await message.answer(f"– {message.text}, назови слово.")

@router.message(Guard.waiting_word, F.text)
async def guard_word(message: Message, state: FSMContext):
    data = await state.get_data()
    if message.text.strip().lower() == "друг":
        await state.clear()
        await message.answer(f"– Проходи, {data['name']}. Дверь 3 открыта.")
        return
    tries = data["tries"] + 1
    if tries >= 3:
        await state.clear()
        await message.answer("– Три ошибки. Уходи. (/doors – начать заново)")
        return
    await state.update_data(tries=tries)
    await message.answer(f"– Не то слово. Осталось попыток: {3 - tries}.")

```

Ловушки порядка

- В aiogram 3 хендлер без фильтра состояния срабатывает в любом состоянии. Стоит эту оказаться выше стража — и оно перехватит все ответы.
- Если /cancel окажется ниже стража, текст «/cancel» уйдёт стражу как неверный пароль, и сбегать не получится.

Где живут состояния: по умолчанию в памяти процесса. Перезапустил бота — все стражи забыли, кто перед ними. Для курса это нормально. В больших ботах состояния хранят в Redis.

Ключ 3. Нажми «открыть» у двери 3, назови стражу своё имя и посмотри в консоль, где запущен бот. Строка, которую напечатал `print(await state.get_state())`, — ключ 3. После этого `print` можно удалить.

Домашка по желанию: пусть /cancel отвечает «Ты и так ни у какой двери», если человек не был в состоянии. Подсказка: `await state.get_state()` вернёт `None`.

Урок 4. Память: SQLite

Сегодня бот начнёт запоминать каждый стук и по команде `/stats` скажет, в какую дверь ты стучишь чаще всего. Данные переживут перезапуск.

SQLite — база данных в одном файле: сервер не нужен, модуль `sqlite3` уже встроен в Python. Для бота берём `aiosqlite` — ту же SQLite, но с `await`. Так запрос к диску не замораживает бота для остальных.

```
pip install aiosqlite
```

Добавь `import aiosqlite` в импорты и под `router = Router()`:

```
DB = "kluchnik.db"

async def init_db():
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "CREATE TABLE IF NOT EXISTS visits ("
            "user_id INTEGER, room INTEGER, ts TEXT DEFAULT CURRENT_TIMESTAMP)"
        )
        await db.commit()

async def add_visit(user_id: int, room: int):
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT INTO visits (user_id, room) VALUES (?, ?)", (user_id, room)
        )
        await db.commit()

async def favorite_room(user_id: int):
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT room, COUNT(*) AS c FROM visits WHERE user_id = ? "
            "GROUP BY room ORDER BY c DESC, room LIMIT 1",
            (user_id,),
        )
        return await cur.fetchone()
```

Три правки в старом коде:

1. В `main()` перед `await dp.start_polling(bot)` добавь `await init_db()`.
2. В хендлер `knock` первой строкой добавь `await add_visit(callback.from_user.id, callback_data.room)`.
3. Новый хендлер выше эха:

```
@router.message(Command("stats"))
async def stats(message: Message):
    row = await favorite_room(message.from_user.id)
```

```
if row is None:
    await message.answer("Ты ещё ни в одну дверь не стучал.")
else:
    await message.answer(f"Чаще всего ты стучишь в дверь {row[0]} ({row[1]} раз).")
```

Как это работает

- `?` в запросе — место для значения. Значения передаём отдельно, кортежем. Никогда не вставляй данные пользователя в SQL через f-строку: это SQL-инъекция.
- `(user_id,)` — кортеж из одного элемента. Запятая обязательна.
- `GROUP BY room` собирает строки с одной дверью в группу, `COUNT(*)` считает размер группы. `ORDER BY c DESC` ставит самую большую группу первой, а `room` во второй сортировке решает ничьи: побеждает меньший номер. `LIMIT 1` оставляет одну строку.
- `fetchone()` вернёт кортеж `(room, c)` или `None`, если данных нет.

Ключ 4. Сначала попробуй предсказать ответ в уме. Потом создай файл `key4.py` с кодом ниже и запусти его. Напечатанная строка — ключ 4.

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE visits (user_id INTEGER, room INTEGER)")
db.executemany(
    "INSERT INTO visits VALUES (?, ?)",
    [(1, 3), (1, 5), (1, 3), (2, 5), (1, 8), (2, 2), (1, 5), (2, 8)],
)
room, c = db.execute(
    "SELECT room, COUNT(*) AS c FROM visits "
    "GROUP BY room ORDER BY c DESC, room LIMIT 1"
).fetchone()
print(f"{room}x{c}")
```

`:memory:` — база в оперативной памяти. Она исчезает, как только скрипт закончится.

Домашка по желанию: команда `/top` — три двери, в которые чаще всего стучат все игроки, с количеством стуков.

Урок 5. Middleware и доступ хозяина

Сегодня защищаем бота от флуда, узнаём свой ID и делаем команду, которая работает только для хозяина.

Что такое middleware

Middleware — код, который оборачивает хендлеры: делает что-то до них и после. Логирование, антифлуд, бан-лист, подключение к базе — всё это middleware. Так не приходится копировать одно и то же в каждый хендлер.

В aiogram их два вида:

- `outer` — срабатывает на каждое событие, ещё до проверки фильтров;
- `inner` (просто `middleware`) — срабатывает, только если нашёлся хендлер, чьи фильтры подошли.

Middleware — это класс с методом `__call__(handler, event, data)`. Вызвал внутри `await handler(event, data)` — событие пошло дальше. Не вызвал — событие остановлено.

Антифлуд

Добавь `import time` в импорты, а `BaseMiddleware` — в строку `from aiogram import ...`. Под `router = Router()`:

```
class AntiFlood(BaseMiddleware):
    def __init__(self, delay: float = 0.7):
        self.delay = delay
        self.last: dict[int, float] = {}

    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if user:
            now = time.monotonic()
            if now - self.last.get(user.id, 0) < self.delay:
                return # слишком часто: молча пропускаем
            self.last[user.id] = now
        return await handler(event, data)

router.message.outer_middleware(AntiFlood())
```

`event_from_user` aiogram сам кладёт в `data` — это автор события. `time.monotonic()` — часы, которые не скачут при переводе времени.

Свой ID и доступ хозяина

Хендлер выше эха:

```
@router.message(Command("id"))
async def my_id(message: Message):
```

```
await message.answer(f"Твой ID: {message.from_user.id}")
```

Отправь боту /id и допиши число в .env:

```
OWNER_ID=123456789
```

В bot.py сразу после `load_dotenv()`:

```
OWNER_ID = int(os.environ["OWNER_ID"])
```

Функция рядом с остальными функциями базы и хендлер выше эха:

```
async def totals():
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute("SELECT COUNT(*), COUNT(DISTINCT user_id) FROM visits")
        return await cur.fetchone()

@router.message(Command("admin"), F.from_user.id == OWNER_ID)
async def admin(message: Message):
    visits, users = await totals()
    await message.answer(f"Стуков: {visits}. Гостей: {users}.")
```

Два фильтра через запятую работают как «И». Чужой человек не пройдёт второй фильтр, и его /admin достанется эху. Попроси друга проверить.

Ключ 5. Middleware можно проверить без Telegram: aiogram умеет «скормить» диспетчеру поддельное сообщение. Создай файл `mw_test.py` с кодом ниже. До запуска предскажи, что он напечатает. Потом запусти. Напечатанное слово — ключ 5.

```
import asyncio
from datetime import datetime

from aiogram import BaseMiddleware, Bot, Dispatcher, F, Router
from aiogram.types import Chat, Message, Update, User

log = []

class Mark(BaseMiddleware):
    def __init__(self, before, after):
        self.before, self.after = before, after

    async def __call__(self, handler, event, data):
        log.append(self.before)
        result = await handler(event, data)
        log.append(self.after)
        return result

router = Router()
router.message.outer_middleware(Mark("g", "e"))
```

```

router.message.outer_middleware(Mark("a", "t"))
router.message.middleware(Mark("r", "d"))

@router.message(F.text == "тык")
async def knock(message: Message):
    log.append("o")

def fake(text: str) -> Update:
    return Update(update_id=1, message=Message(
        message_id=1, date=datetime.now(), text=text,
        chat=Chat(id=1, type="private"),
        from_user=User(id=1, is_bot=False, first_name="Test"),
    ))

async def main():
    dp = Dispatcher()
    dp.include_router(router)
    bot = Bot("42:TEST") # фейковый токен: в сеть мы не ходим
    await dp.feed_update(bot, fake("привет"))
    print("".join(log))
    await bot.session.close()

asyncio.run(main())

```

После этого замени «привет» на «тык» и запусти ещё раз. Объясни себе, откуда взялись новые буквы и почему они встали именно в середину.

Домашка по желанию: middleware, который пишет в консоль, кто и что прислал боту.

Урок 6. Бот 24/7: сервер и systemd

Сегодня переносим бота на сервер, чтобы он работал без твоего компьютера, и учимся читать его логи.

Подготовка проекта

Файл `.gitignore` в папке проекта:

```
venv/  
.env  
*.db  
__pycache__/
```

Список зависимостей: `pip freeze > requirements.txt`. Залей код в приватный репозиторий на GitHub. Файл `.env` туда не попадёт — на сервере создашь его заново.

Сервер

Подойдёт самый дешёвый VPS с Ubuntu 24.04. Зайди на него по SSH и выполни:

```
sudo apt update  
sudo apt install -y python3-venv git  
git clone https://github.com/ТВОЙ_ЛОГИН/kluchnik.git ~/kluchnik  
cd ~/kluchnik  
python3 -m venv venv  
venv/bin/pip install -r requirements.txt  
nano .env
```

Для приватного репозитория GitHub спросит логин и токен доступа (Personal access token). В `.env` впиши те же `BOT_TOKEN` и `OWNER_ID`, что дома. Проверь запуск руками: `venv/bin/python bot.py`, затем `Ctrl+C`.

systemd: бот как служба

Узнай своё имя пользователя (`whoami`) и путь к папке (`pwd`). Создай файл службы: `sudo nano /etc/systemd/system/kluchnik.service`.

```
[Unit]  
Description=Kluchnik bot  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
User=ТВОЙ_ПОЛЬЗОВАТЕЛЬ  
WorkingDirectory=/home/ТВОЙ_ПОЛЬЗОВАТЕЛЬ/kluchnik  
ExecStart=/home/ТВОЙ_ПОЛЬЗОВАТЕЛЬ/kluchnik/venv/bin/python bot.py  
Restart=always  
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Если заходишь под root, путь будет /root/kluchnik, а пользователь — root. Запускаем:

```
sudo systemctl daemon-reload
sudo systemctl enable --now kluchnik
sudo systemctl status kluchnik
journalctl -u kluchnik -f
```

- `Restart=always` : упал бот — systemd поднимет его через 5 секунд.
- `enable` : бот стартует сам после перезагрузки сервера.
- `journalctl -f` показывает живой лог. Выход — Ctrl+C.
- Обновить код: `cd ~/kluchnik && git pull && sudo systemctl restart kluchnik`.

Polling или webhook? При webhook Telegram сам стучится на твой сервер по HTTPS. Для этого нужны домен и сертификат. Для бота такого размера polling проще и надёжнее.

Один токен — один запущенный бот

Telegram отдаёт обновления только одному процессу. Запустишь второй с тем же токеном — они начнут драться за обновления. Это самая частая беда: бот уже работает на сервере, а ты запускаешь его ещё и дома для отладки.

Ключ 6. Пока бот работает на сервере, запусти его же у себя на компьютере: `python bot.py`. Подожди 10–20 секунд и отправь боту пару сообщений. В одной из консолей (дома или на сервере в `journalctl -u kluchnik -f`) появится ошибка `Failed to fetch updates - TelegramConflictError: Telegram server says - ...`. Первое слово после «Telegram server says -», маленькими буквами, — ключ 6. Потом останови домашнего бота.

Домашка по желанию: перезагрузи сервер (`sudo reboot`) и проверь, что бот поднялся сам.

Урок 7. Финал: пропуск у Привратника

Сегодня твой бот сам заработает тебе пропуск в чат выпускников.

Как проходит экзамен

1. Пишешь Привратнику `/gate`. Он выдаёт одноразовое слово из 8 символов, оно живёт 15 минут.
2. Отправляешь своему боту `/proof СЛОВО`.
3. Бот считает код из шести ключей, твоего ID, ID бота и слова и отвечает: `PROOF слово КОД`.
4. Пересылаешь этот ответ Привратнику. Именно «Переслать», а не скопировать текст.
5. Привратник проверяет код и выдаёт одноразовую ссылку в чат выпускников.

Почему это не подделать:

- Код зависит от твоего Telegram ID. Ответ чужого бота для тебя не сработает.
- Привратник видит, от какого бота переслано сообщение, и сам знает его ID.
- Один бот проводит через дверь только одного человека.
- Слово одноразовое и быстро протухает.

Код

Добавь `import hashlib` в импорты. После строки с `OWNER_ID`:

```
KEYS = ["ключ1", "ключ2", "ключ3", "ключ4", "ключ5", "ключ6"]
```

Подставь свои ключи из `keys.txt` в порядке уроков. Регистр и знаки — точно как их выдали.

Функцию — к остальным функциям, хендлер — выше эха:

```
def make_proof(user_id: int, bot_id: int, word: str) -> str:
    raw = "|".join([*KEYS, str(user_id), str(bot_id), word])
    return hashlib.sha256(raw.encode()).hexdigest()[:10].upper()

@router.message(Command("proof"), F.from_user.id == OWNER_ID)
async def proof(message: Message, command: CommandObject, bot: Bot):
    if not command.args:
        await message.answer("Формат: /proof СЛОВО_ОТ_ПРИВРАТНИКА")
        return
    word = command.args.strip()
    code = make_proof(message.from_user.id, bot.id, word)
    await message.answer(f"PROOF {word} {code}")
```

Как это работает

- `"|".join(...)` склеивает ключи, твой ID, ID бота и слово через вертикальную черту.
- `sha256` — хеш-функция: из любой строки получается 64 символа. Поменяй на входе один символ — изменится весь хеш. Обратно хеш не разворачивается, поэтому по чужому коду

ключи не узнать.

- `[:10].upper()` — первые 10 символов, заглавными буквами.
- `bot.id` — ID бота, то самое число из токена (урок 1).
- `F.from_user.id == OWNER_ID` — пропуск выдаётся только хозяину бота (урок 5).

Обнови бота на сервере (урок 6) и иди к Привратнику.

Если Привратник не пускает

- «Код не сходится» — проверь ключи: порядок, регистр, лишние пробелы и кавычки. Какой ключ неверный, Привратник не скажет.
- «Слово протухло» — возьми новое: `/gate`.
- «Это копия» — нужно переслать сообщение, а не копировать его текст.
- «Этот бот уже провёл другого человека» — нужен свой бот.
- Бот отвечает «Ты сказал: `/proof ...`» — хендлер `/proof` стоит ниже эха или в `.env` записан не твой `OWNER_ID`.

Полный `bot.py`

Сверь с ним свой файл, если что-то не работает. Вместо «ключ1»...«ключ6» — твои ключи.

```
import asyncio
import hashlib
import logging
import os
import time

import aiosqlite
from aiogram import BaseMiddleware, Bot, Dispatcher, F, Router
from aiogram.filters import Command, CommandObject, CommandStart
from aiogram.filters.callback_data import CallbackData
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import CallbackQuery, Message
from aiogram.utils.keyboard import InlineKeyboardBuilder
from dotenv import load_dotenv

load_dotenv()
OWNER_ID = int(os.environ["OWNER_ID"])
DB = "kluchnik.db"
KEYS = ["ключ1", "ключ2", "ключ3", "ключ4", "ключ5", "ключ6"]

router = Router()

# ----- урок 4: память -----
async def init_db():
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "CREATE TABLE IF NOT EXISTS visits ("
            "user_id INTEGER, room INTEGER, ts TEXT DEFAULT CURRENT_TIMESTAMP)"
        )
        await db.commit()
```

```

async def add_visit(user_id: int, room: int):
    async with aiosqlite.connect(DB) as db:
        await db.execute(
            "INSERT INTO visits (user_id, room) VALUES (?, ?)", (user_id, room)
        )
        await db.commit()

async def favorite_room(user_id: int):
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute(
            "SELECT room, COUNT(*) AS c FROM visits WHERE user_id = ? "
            "GROUP BY room ORDER BY c DESC, room LIMIT 1",
            (user_id,),
        )
        return await cur.fetchone()

async def totals():
    async with aiosqlite.connect(DB) as db:
        cur = await db.execute("SELECT COUNT(*), COUNT(DISTINCT user_id) FROM visits")
        return await cur.fetchone()

# ----- урок 2: двери -----
class Door(CallbackData, prefix="kd"):
    room: int
    act: str

def corridor():
    kb = InlineKeyboardBuilder()
    for room in (1, 2, 3):
        kb.button(text=f"Дверь {room}: постучать", callback_data=Door(room=room, act="knock"))
        kb.button(text="открыть", callback_data=Door(room=room, act="open"))
    kb.adjust(2)
    return kb.as_markup()

# ----- урок 3: страж -----
class Guard(StatesGroup):
    waiting_name = State()
    waiting_word = State()

# ----- урок 5: антифлуд -----
class AntiFlood(BaseMiddleware):
    def __init__(self, delay: float = 0.7):
        self.delay = delay
        self.last: dict[int, float] = {}

    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if user:
            now = time.monotonic()
            if now - self.last.get(user.id, 0) < self.delay:

```

```

        return # слишком часто: молча пропускаем
        self.last[user.id] = now
        return await handler(event, data)

router.message.outer_middleware(AntiFlood())

# ----- упок 7: пропуск -----
def make_proof(user_id: int, bot_id: int, word: str) -> str:
    raw = "|".join([*KEYS, str(user_id), str(bot_id), word])
    return hashlib.sha256(raw.encode()).hexdigest()[:10].upper()

# ----- хендлеры: порядок важен -----
@router.message(CommandStart(deep_link=True))
async def start_with_word(message: Message, command: CommandObject):
    await message.answer(command.args[:-1])

@router.message(CommandStart())
async def start(message: Message):
    await message.answer("Ключник на посту. Двери пока закрыты.")

@router.message(Command("id"))
async def my_id(message: Message):
    await message.answer(f"Твой ID: {message.from_user.id}")

@router.message(Command("cancel"))
async def cancel(message: Message, state: FSMContext):
    await state.clear()
    await message.answer("Ты отошёл от двери.")

@router.message(Command("doors"))
async def doors(message: Message):
    await message.answer("Коридор. Три двери.", reply_markup=corridor())

@router.message(Command("stats"))
async def stats(message: Message):
    row = await favorite_room(message.from_user.id)
    if row is None:
        await message.answer("Ты ещё ни в одну дверь не стучал.")
    else:
        await message.answer(f"Чаще всего ты стучишь в дверь {row[0]} ({row[1]} раз).")

@router.message(Command("admin"), F.from_user.id == OWNER_ID)
async def admin(message: Message):
    visits, users = await totals()
    await message.answer(f"Стуков: {visits}. Гостей: {users}.")

@router.message(Command("proof"), F.from_user.id == OWNER_ID)
async def proof(message: Message, command: CommandObject, bot: Bot):

```

```

if not command.args:
    await message.answer("Формат: /proof СЛОВО_ОТ_ПРИВРАТНИКА")
    return
word = command.args.strip()
code = make_proof(message.from_user.id, bot.id, word)
await message.answer(f"PROOF {word} {code}")

@router.callback_query(Door.filter(F.act == "knock"))
async def knock(callback: CallbackQuery, callback_data: Door):
    await add_visit(callback.from_user.id, callback_data.room)
    await callback.answer(f"За дверью {callback_data.room} тихо.")

@router.callback_query(Door.filter(F.act == "open"))
async def open_door(callback: CallbackQuery, callback_data: Door, state: FSMContext):
    if callback_data.room != 3:
        await callback.answer("Заперто.", show_alert=True)
        return
    await state.set_state(Guard.waiting_name)
    await callback.message.edit_text("У двери 3 стоит страж.\n— Кто идёт?")
    await callback.answer()

@router.message(Guard.waiting_name, F.text)
async def guard_name(message: Message, state: FSMContext):
    await state.update_data(name=message.text, tries=0)
    await state.set_state(Guard.waiting_word)
    await message.answer(f"— {message.text}, назови слово.")

@router.message(Guard.waiting_word, F.text)
async def guard_word(message: Message, state: FSMContext):
    data = await state.get_data()
    if message.text.strip().lower() == "друг":
        await state.clear()
        await message.answer(f"— Проходи, {data['name']}. Дверь 3 открыта.")
        return
    tries = data["tries"] + 1
    if tries >= 3:
        await state.clear()
        await message.answer(f"— Три ошибки. Уходи. (/doors — начать заново)")
        return
    await state.update_data(tries=tries)
    await message.answer(f"— Не то слово. Осталось попыток: {3 - tries}.")

@router.message(F.text)
async def echo(message: Message):
    await message.answer(f"Ты сказал: {message.text}")

async def main():
    logging.basicConfig(level=logging.INFO)
    await init_db()
    bot = Bot(os.environ["BOT_TOKEN"])
    dp = Dispatcher()
    dp.include_router(router)

```

```
await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```

Ты прошёл весь путь: от токена у BotFather до бота, который сам живёт на сервере. Что изучать дальше: webhook, хранение состояний в Redis, платежи и мини-приложения. Документация aiogram: docs.aiogram.dev.